

```
.card {
  background: var(--surface);
  border-radius: 16px;
  box-shadow: 0 10px 30px rgba(0,0,0,.08);
}

.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}

@media (min-width: 700px) {
  .hero h1 { font-size: clamp(2rem, 5vw, 4rem); }
}

:root { --primary: #2563eb; --radius: 12px; }
.button:hover { transform: translateY(-2px); }
@keyframes pulse { 50% { transform: scale(1.05); } }
```

BEGINNER → INTERMEDIATE → ADVANCED

Complete CSS3 Mastery Guide

A structured, project-based path through modern CSS — layout, responsive design, motion, and maintainable architecture — with authentic references to the official specifications and documentation professional developers actually use.

FLEXBOX & GRID

RESPONSIVE DESIGN

MODERN CSS

ACCESSIBLE UI

24 Chapters

Fundamentals to full projects

Official Sources

W3C CSSWG · MDN · web.dev

Method

Read → Type → Run → Inspect → Modify

Welcome — How This Guide Works

This edition has been redesigned end-to-end: every chapter is expanded with deeper explanations, additional real-world examples, styled code previews, accessibility and performance notes, and links to authentic official documentation (the W3C CSS Working Group specifications, MDN Web Docs, and web.dev) so you can always verify details at the source.

1

READ THE
CONCEPT

2

TYPE THE
EXAMPLE

3

RUN IN BROWSER

4

INSPECT THE
RESULT

5

MODIFY &
EXPERIMENT

Layer	Responsibility	Typical Job	Official Spec
HTML	Structure & meaning	Headings, forms, landmarks, images	html.spec.whatwg.org
CSS	Presentation	Color, layout, spacing, typography	w3.org/Style/CSS
JavaScript	Behavior & logic	Events, DOM updates, API calls	tc39.es/ecma262

Table of Contents

PART I — FOUNDATIONS

- [01 CSS Fundamentals](#)
- [02 Selectors, Combinators & Specificity](#)
- [03 Colors, Units & Functions](#)
- [04 Typography](#)
- [05 Box Model & Sizing](#)
- [06 Display, Visibility & Overflow](#)

PART II — LAYOUT

- [07 Positioning & Stacking](#)
- [08 Flexbox](#)
- [09 CSS Grid](#)
- [10 Responsive Design & Media Queries](#)

PART III — VISUAL DESIGN

- [11 Backgrounds, Borders, Shadows & Gradients](#)
- [12 Images, Object Fit & Aspect Ratio](#)
- [13 Pseudo-classes & Pseudo-elements](#)
- [14 Forms & UI States](#)

PART IV — MOTION & MODERN CSS

- [15 Transitions, Transforms & Animations](#)
- [16 CSS Variables & Modern Functions](#)
- [17 Cascade, Inheritance & Specificity in Depth](#)

PART V — QUALITY & PRACTICE

- [18 CSS Architecture & Maintainability](#)
- [19 Accessibility, Performance & Security](#)
- [20 Complete Responsive Website Example](#)
- [21 Mini Projects](#)
- [22 Debugging & Common Mistakes](#)

PART VI — REFERENCE

- [23 CSS Cheat Sheet](#)
- [24 Practice Questions & Final Checklist](#)

APPENDIX

- [A CSS Timeline & History](#)
- [B Official Resources & Further Reading](#)



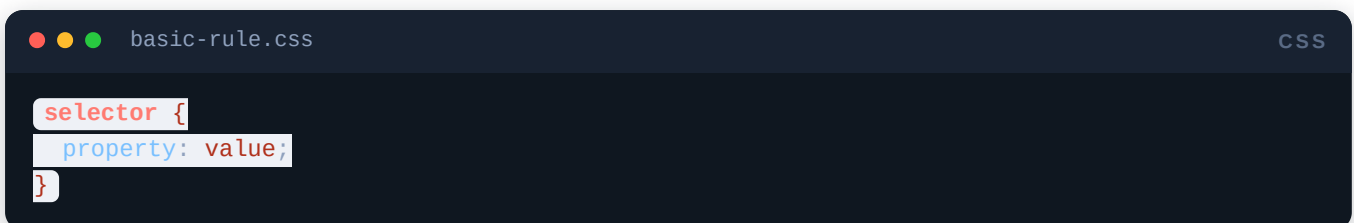
CSS (Cascading Style Sheets) describes how HTML content is presented: color, typography, spacing, sizing, layout, responsive behavior, and visual effects. **HTML** provides structure and semantics, **CSS** provides presentation, and **JavaScript** commonly provides behavior and application logic.

Why CSS Is Important

- ☐ Separates content from presentation.
- ☐ Creates reusable visual rules across an entire site.
- ☐ Supports responsive design for every screen size.
- ☐ Provides Flexbox and Grid for real layout systems.
- ☐ Adds states, transitions, transforms, and animations.
- ☐ Makes visual consistency far easier to maintain.

Three Ways to Add CSS

Method	How	When to use it
Inline	<code>style</code> attribute on an element	Rare one-off overrides, quick debugging
Internal	A <code><style></code> element in the document head	Small single-page demos
External	A separate file linked with <code><link rel="stylesheet"></code>	Real projects — cacheable, reusable, maintainable



A rule is made of a **selector** (what to target) and a **declaration block** of `property: value;` pairs wrapped in curly braces.



External stylesheets are the professional default

They separate concerns, are cached by the browser across page loads, and can be shared across every page of a site from one single source of truth.

W3C — Cascading Style Sheets (current work)

<https://www.w3.org/Style/CSS/current-work>

The W3C CSS Working Group's official index of specifications.

MDN — CSS: Cascading Style Sheets

<https://developer.mozilla.org/en-US/docs/Web/CSS>

Mozilla's complete CSS reference, guides, and tutorials.

web.dev — Learn CSS

<https://web.dev/learn/css>



Selector	Example	Matches
Type selector	<code>p</code>	Matches every element of that tag
Class selector	<code>.card</code>	Matches every element with that class
ID selector	<code>#header</code>	Matches the single element with that id
Universal selector	<code>*</code>	Matches every element
Attribute selector	<code>input[type="email"]</code>	Matches elements with a given attribute/value
Descendant	<code>.nav a</code>	Any a anywhere inside .nav
Child	<code>.nav > a</code>	Only direct children
Adjacent sibling	<code>h2 + p</code>	The p immediately after an h2
General sibling	<code>h2 ~ p</code>	Any p that follows an h2 at the same level

Grouping Selectors

```
grouping.css
CSS
h1, h2, h3 {
  font-family: system-ui, sans-serif;
}
```

Specificity

Specificity helps resolve competing declarations that target the same element. In simplified terms: **IDs** are more specific than **classes**, **attributes**, and **pseudo-classes**, which are more specific than **element** and **pseudo-element** selectors. Cascade origin, importance, layers, and source order also matter and are covered in depth in Chapter 17.



!important is not the default fix

Reaching for !important treats a symptom, not the cause. Open DevTools, inspect the matched rules, and determine why the declaration you want is losing — almost always a specificity or source-order issue you can fix cleanly instead.

MDN — CSS selectors

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_selectors

Complete reference for every selector and combinator type.

MDN — Specificity

<https://developer.mozilla.org/en-US/docs/Web/CSS/Specificity>



CSS supports named colors, hexadecimal colors, `rgb()` / `rgba()`, `hsl()` / `hsla()`, and newer color functions like `oklch()` for wider, more perceptually uniform color gamuts.

Unit	Definition	Best for
<code>px</code>	CSS pixel	Borders and fine, non-scaling details
<code>%</code>	Relative to a reference dimension	Fluid widths relative to a parent
<code>em</code>	Relative to the element's own font size	Spacing that should scale with local text
<code>rem</code>	Relative to the root font size	Scalable, predictable typography and spacing
<code>vw</code> / <code>vh</code>	Viewport width / height	Full-bleed hero sections, viewport-aware type
<code>vmin</code> / <code>vmax</code>	Smaller / larger viewport dimension	Consistent sizing across orientations
<code>ch</code>	Width of the "0" character	Readable text-column measures
<code>fr</code>	Fractional Grid track	Flexible Grid column/row sizing

Useful Functions

functions.css

CSS

```
width: min(90%, 1100px);
font-size: clamp(1rem, 2vw, 1.5rem);
padding: calc(1rem + 1vw);
grid-template-columns: repeat(3, 1fr);
```



Pick units by purpose, not habit

Use rem/em for scalable typography and spacing, percentages and fr for layout proportions, and viewport-aware values (vw, clamp) only where they genuinely improve responsiveness — overusing viewport units can make text illegible at extreme sizes.

MDN — CSS values and units

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Values_and_Units

Every unit type with browser support and worked examples.

MDN — color function docs

https://developer.mozilla.org/en-US/docs/Web/CSS/color_value

Full reference for every valid CSS color syntax.



Property	Purpose
<code>font-family</code>	Font stack / fallback chain
<code>font-size</code>	Text size
<code>font-weight</code>	Thickness
<code>font-style</code>	Italic / normal
<code>line-height</code>	Vertical rhythm between lines
<code>letter-spacing</code>	Character spacing
<code>text-align</code>	Horizontal alignment
<code>text-decoration</code>	Underline, strikethrough, etc.
<code>text-transform</code>	Case conversion (uppercase, capitalize)
<code>overflow-wrap</code> / <code>text-overflow</code>	Handling long, unbreakable content

Responsive Typography

```
typography.css CSS

body {
  font-family: system-ui, sans-serif;
  font-size: 1rem;
  line-height: 1.6;
}

h1 {
  font-size: clamp(2rem, 5vw, 4rem);
  line-height: 1.05;
}
```

Using `clamp()` for headline sizing lets a heading scale smoothly between a sensible minimum and maximum as the viewport changes, without needing a separate media query for every breakpoint.



Loading web fonts responsibly

Web fonts can be loaded with `@font-face` or a provider like Google Fonts. Limit the number of weights and styles you load, and always provide a useful system-font fallback so text still renders instantly while the custom font downloads.

MDN — CSS text and font styling

https://developer.mozilla.org/en-US/docs/Learn/CSS/Styling_text

Official guide to font and text properties with live examples.

MDN — @font-face

<https://developer.mozilla.org/en-US/docs/Web/CSS/@font-face>



A normal CSS box is conceptually made of four layers, from the inside out:

Layer	What it is
Content	The actual text, image, or child content
Padding	Space inside the border, around the content
Border	The boundary line around the padding
Margin	Space outside the border, separating this box from others

box-sizing: The Recommended Reset

reset.css

CSS

```
*, *::before, *::after {  
  box-sizing: border-box;  
}
```

With `border-box`, a declared `width` and `height` already include padding and border, which usually makes sizing far easier to reason about than the default `content-box`.



Margin collapsing

Certain vertical margins between block-level siblings (and between a parent and its first/last child) can collapse into a single margin in normal flow. Flexbox and Grid create different formatting contexts and don't have this behavior for their direct children.

MDN — The box model

https://developer.mozilla.org/en-US/docs/Learn/CSS/Building_blocks/The_box_model

Visual, official explanation of content/padding/border/margin.

MDN — box-sizing

<https://developer.mozilla.org/en-US/docs/Web/CSS/box-sizing>

Full reference for content-box vs border-box.



Value	Behavior
<code>block</code>	Starts a new line; accepts width/height/margin fully
<code>inline</code>	Flows with surrounding text; width/height mostly ignored
<code>inline-block</code>	Flows inline but accepts box dimensions
<code>none</code>	Removed from layout entirely
<code>flex</code>	Creates a flex formatting context for children
<code>grid</code>	Creates a grid formatting context for children



visibility vs display

`visibility: hidden` normally preserves the element's layout space (it's invisible but still “takes up room”), while `display: none` removes it from layout entirely, as if it were never there.

Overflow

overflow.css

CSS

```
.panel {  
  max-height: 300px;  
  overflow: auto;  
}
```

Common overflow values are `visible`, `hidden`, `clip`, `auto`, and `scroll`. Be careful with `hidden` — it's an easy way to accidentally clip important content that a user needed to scroll to see.

MDN — display

<https://developer.mozilla.org/en-US/docs/Web/CSS/display>

Full reference for every display value, including modern two-value syntax.

MDN — overflow

<https://developer.mozilla.org/en-US/docs/Web/CSS/overflow>

Complete guide to overflow behavior and scroll containers.



Value	Behavior
<code>static</code>	Normal flow; the default for every element
<code>relative</code>	Stays in flow, can be nudged with an offset
<code>absolute</code>	Removed from flow, positioned against a containing block
<code>fixed</code>	Typically positioned relative to the viewport, ignores scrolling
<code>sticky</code>	Acts relative until a scroll threshold, then sticks within its container

A Positioned Badge

```
badge.css

.card { position: relative; }

.badge {
  position: absolute;
  top: 1rem;
  right: 1rem;
}
```

z-index & Stacking Contexts

`z-index` controls stacking order where it applies (positioned elements, flex/grid items, and a few other cases). Several CSS features — `opacity < 1`, `transform`, `filter`, and others — create new **stacking contexts**, so a very large `z-index` does not automatically place an element above everything else on the page.



Debugging z-index issues

When an element seems “stuck behind” another despite a high `z-index`, check whether an ancestor has created its own stacking context. The `z-index` only competes with siblings inside the same stacking context.

MDN — position

<https://developer.mozilla.org/en-US/docs/Web/CSS/position>

Full reference for every positioning scheme.

MDN — Stacking context

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_positioned_layout/Understanding_z-index/The_stacking_context

How stacking contexts are created and how `z-index` really works.



Flexbox is ideal for laying out a single row or column of items that need alignment and flexible sizing — navigation bars, button groups, card rows, and centering content.

Container Properties

- ☐ `display: flex`
- ☐ `flex-direction`
- ☐ `justify-content`
- ☐ `align-items`
- ☐ `align-content`
- ☐ `flex-wrap`
- ☐ `gap`

Item Properties

- ☐ `flex-grow`
- ☐ `flex-shrink`
- ☐ `flex-basis`
- ☐ `flex` (shorthand)
- ☐ `align-self`
- ☐ `order` — use carefully, since visual order can differ from reading/DOM order

Responsive Cards With Flexbox

```
flex-cards.css CSS

.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}

.card {
  flex: 1 1 260px;
}
```

`flex: 1 1 260px` means: grow to fill space, shrink if needed, but start from a 260px basis — a compact way to get responsive, wrapping cards with zero media queries.

MDN — Basic concepts of flexbox

https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Flexbox

Official, illustrated introduction to the flex model.

CSS-Tricks — A Complete Guide to Flexbox

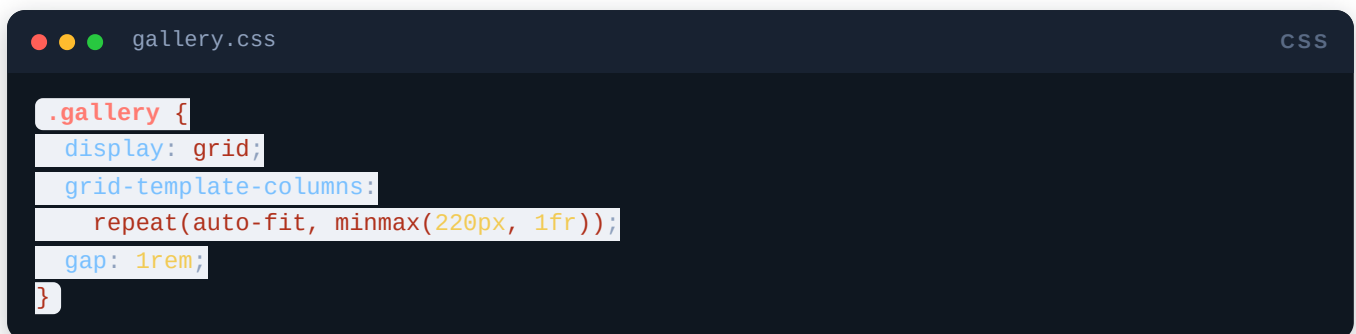
<https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

The most widely bookmarked visual reference for every flex property.



Feature	Purpose
<code>grid-template-columns</code> / <code>rows</code>	Define the tracks (columns/rows)
<code>gap</code>	Row and column spacing
<code>fr</code>	Fractional track unit
<code>repeat()</code>	Repeated tracks without retyping
<code>minmax()</code>	A minimum and maximum size for a track
<code>auto-fit</code> / <code>auto-fill</code>	Responsive, self-adjusting track patterns
<code>grid-column</code> / <code>grid-row</code>	Item placement and spanning
<code>grid-template-areas</code>	Named layout regions

A Fully Responsive Grid, No Media Queries



`auto-fit` plus `minmax()` lets the browser decide how many 220px-minimum columns fit the available width, then stretches them evenly to fill the remaining space.

Flexbox vs. Grid

	Flexbox	Grid
Dimensions	One-dimensional (a row OR a column)	Two-dimensional (rows AND columns together)
Best for	Component-level rows/columns, alignment, nav bars	Page-level layout, dashboards, galleries
Sizing model	Content pushes/pulls along one axis	Explicit tracks you define up front



They're not rivals

Real layouts commonly nest the two: Grid for the overall page shell, Flexbox inside individual grid cells for aligning that cell's own content.

MDN — CSS Grid Layout

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout

The complete official guide to every Grid property and concept.

Grid by Example (Rachel Andrew)

<https://gridbyexample.com/>

A widely cited, example-driven reference maintained by a CSS Working Group Grid spec editor.



Responsive design adapts layouts to different viewport sizes and user preferences. A **mobile-first** strategy starts with the base, single-column layout and adds enhancements as more space becomes available — rather than starting wide and cramming things down.

```
mobile-first.css

.cards { grid-template-columns: 1fr; }

@media (min-width: 700px) {
  .cards { grid-template-columns: repeat(2, 1fr); }
}

@media (min-width: 1000px) {
  .cards { grid-template-columns: repeat(3, 1fr); }
}
```

Useful Media Features Beyond Width

Feature	What it detects
<code>prefers-color-scheme</code>	Light or dark mode preference
<code>prefers-reduced-motion</code>	Motion sensitivity preference
<code>hover</code> / <code>pointer</code>	Whether the input can hover, and how precise it is
<code>orientation</code>	Portrait or landscape

```
reduced-motion.css

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```



Respect reduced motion by default

Wrapping your animation and transition durations in a `prefers-reduced-motion` query isn't optional polish — for some users, unrestrained motion can trigger real physical discomfort or vestibular symptoms.

MDN — Using media queries

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_media_queries/Using_media_queries

Complete official reference for every media feature.

web.dev — prefers-reduced-motion

<https://web.dev/prefers-reduced-motion/>



Background properties include `background-color`, `background-image`, `background-repeat`, `background-position`, `background-size`, `background-attachment`, and the `background` shorthand that combines them.

A Darkened Hero Image

```
hero.css
CSS

.hero {
  background:
    linear-gradient(rgba(0,0,0,.55), rgba(0,0,0,.55)),
    url("hero.jpg") center / cover no-repeat;
}
```

Layering a semi-transparent gradient over a background image is a reliable way to guarantee text contrast on top of a photo, regardless of the photo's own colors.

Border & Shadow

```
card-style.css
CSS

.card {
  border: 1px solid #d9dee7;
  border-radius: 16px;
  box-shadow: 0 10px 30px rgba(0,0,0,.08);
}
```



Three kinds of gradient

Gradients can be `linear-gradient()`, `radial-gradient()`, or `conic-gradient()`, and are commonly used for backgrounds, overlays, and decorative accents without needing an extra image file.

MDN — Using CSS gradients

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_images/Using_CSS_gradients

Official guide with live examples of all three gradient types.

MDN — box-shadow

<https://developer.mozilla.org/en-US/docs/Web/CSS/box-shadow>

Full syntax reference, including multiple layered shadows.



CSS controls how replaced elements — images, videos, and similar embedded content — fit into the box you've given them, independent of their original file dimensions.

```
media.css

.media {
  width: 100%;
  aspect-ratio: 16 / 9;
  object-fit: cover;
  object-position: center;
}
```

object-fit value	Behavior
<code>cover</code>	Fills the box completely, preserving aspect ratio; may crop the edges
<code>contain</code>	Keeps the full image visible, preserving aspect ratio; may leave empty space
<code>fill</code>	Stretches to fill the box exactly, ignoring aspect ratio
<code>none</code>	Uses the image's natural size, ignoring the box



Reserve space to prevent layout shift

Setting an explicit aspect-ratio (or HTML width/height attributes) lets the browser reserve the correct space before the image finishes downloading, preventing the rest of the page from jumping around — a key factor in Core Web Vitals.

MDN — object-fit

<https://developer.mozilla.org/en-US/docs/Web/CSS/object-fit>

Full reference with visual comparisons of each value.

MDN — aspect-ratio

<https://developer.mozilla.org/en-US/docs/Web/CSS/aspect-ratio>

How to reserve consistent proportions for media and boxes.



Pseudo-classes describe a state or condition of an existing element:

Pseudo-class	Matches
<code>:hover</code>	Pointer is over the element
<code>:focus</code> / <code>:focus-visible</code>	Element has keyboard/programmatic focus
<code>:active</code>	Element is being activated (e.g. mouse down)
<code>:checked</code>	A checked checkbox/radio/option
<code>:disabled</code>	A disabled form control
<code>:first-child</code>	The first child of its parent
<code>:nth-child()</code>	Pattern-based positional selection
<code>:not()</code>	Negation — excludes a selector
<code>:is()</code> / <code>:where()</code>	Groups selectors; <code>:where()</code> carries zero specificity

Pseudo-elements

```
arrow-button.css
.button::before {
  content: "→";
  margin-right: .4rem;
}
```

Common pseudo-elements include `::before`, `::after`, `::first-letter`, `::first-line`, and `::selection`. They generate or target content that isn't a real DOM node.



Keep meaning in HTML

Generated content via `::before`/`::after` is invisible to some assistive technology and to copy-paste. Essential semantic information — anything the user needs to read or reference — should normally remain in real HTML, not be injected purely through CSS.

MDN — Pseudo-classes

<https://developer.mozilla.org/en-US/docs/Web/CSS/Pseudo-classes>

Complete list of every standard pseudo-class.

MDN — Pseudo-elements

<https://developer.mozilla.org/en-US/docs/Web/CSS/Pseudo-elements>

Complete list of every standard pseudo-element.



Good form styling includes clear labels, a visible focus state, readable validation feedback, and adequately sized touch/click targets.

```
form-field.css

.field input {
  padding: .75rem 1rem;
  border: 1px solid #b8c2cc;
  border-radius: 8px;
}

.field input:focus-visible {
  outline: 3px solid #7aa7ff;
  outline-offset: 2px;
}
```

State	Selector
Validation state	<code>:valid</code> / <code>:invalid</code>
Required control	<code>:required</code>
Availability	<code>:disabled</code> / <code>:enabled</code>
Checkbox/radio state	<code>:checked</code>



Never remove focus outlines without a replacement

Deleting the default outline for aesthetics, without providing an equally visible custom focus style, makes a site unusable for anyone navigating by keyboard. Always design a clear, high-contrast `:focus-visible` style instead.

MDN — UI pseudo-classes

https://developer.mozilla.org/en-US/docs/Web/CSS/Pseudo-classes#uiinput_state_pseudo_classes

Reference for `:valid`, `:invalid`, `:required`, `:checked`, and related states.

MDN — `:focus-visible`

<https://developer.mozilla.org/en-US/docs/Web/CSS/:focus-visible>

Why `focus-visible` exists and how it differs from `:focus`.



Transitions

Transitions interpolate a property smoothly between its old and new value whenever that value changes.

```
hover-lift.css

.button {
  transition: transform .2s ease, box-shadow .2s ease;
}

.button:hover {
  transform: translateY(-2px);
}
```

Transforms

Transform functions include `translate()`, `scale()`, `rotate()`, `skew()`, and the `transform-origin` property that controls their pivot point.

Keyframe Animations

```
pulse.css

@keyframes pulse {
  0%, 100% { transform: scale(1); }
  50% { transform: scale(1.05); }
}

.logo {
  animation: pulse 2s ease-in-out infinite;
}
```



Animate transform and opacity

These two properties can typically be updated by the browser's compositor without triggering an expensive layout recalculation, making them the most efficient properties to animate. Always wrap non-essential motion in a `prefers-reduced-motion` query.

MDN — Using CSS transitions

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_transitions/Using_CSS_transitions

Official guide to transition properties and timing functions.

MDN — CSS animations

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_animations/Using_CSS_animations

Full walkthrough of `@keyframes` and the animation shorthand.

web.dev — Animations guide

<https://web.dev/animations-guide/>



Custom properties (informally, “CSS variables”) are reusable values declared once and referenced anywhere, and unlike preprocessor variables, they participate fully in the cascade and can change per element, per media query, or even via JavaScript at runtime.

```
variables.css CSS

:root {
  --primary: #2563eb;
  --space: .75rem;
  --radius: 12px;
}

.button {
  background: var(--primary);
  padding: var(--space) 1rem;
  border-radius: var(--radius);
}
```

Fallback Values

```
fallback.css CSS

.text { color: var(--text-color, #222); }
```

`var(--text-color, #222)` uses `#222` whenever `--text-color` is undefined, which makes components safe to reuse even before a theme defines every token.

Function	Purpose
<code>calc()</code>	Mixed-unit calculations, e.g. <code>100% - 2rem</code>
<code>min()</code>	Uses whichever value is smaller
<code>max()</code>	Uses whichever value is larger
<code>clamp()</code>	A minimum, a preferred, and a maximum value together
<code>minmax()</code>	A size range for a Grid track
<code>var()</code>	Reads a custom property, with an optional fallback

MDN — Using CSS custom properties

https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties

Official guide to declaring and using custom properties.

MDN — CSS math functions

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Values_and_Units/CSS_Value_Functions



The cascade resolves competing declarations using, roughly in order of precedence: origin and `!important`, cascade layers, specificity, and finally source order (later wins when everything else ties).

Inheritance

Many text-related properties (color, font, line-height, etc.) inherit automatically from parent to child. Box-model properties — margin, padding, border, and width — generally do **not** inherit.

Global Values

Value	Effect
<code>inherit</code>	Explicitly takes the parent's computed value
<code>initial</code>	Resets to the property's specification default
<code>unset</code>	Acts like inherit for inherited properties, initial otherwise
<code>revert</code>	Rolls back to the browser's built-in default style
<code>revert-layer</code>	Rolls back to the value from a previous cascade layer

Cascade Layers

```
layers.css CSS

@layer reset, base, components, utilities;

@layer base {
  body { margin: 0; }
}

@layer components {
  .card { padding: 1rem; }
}
```

Layers declared earlier in `@layer reset, base, components, utilities;` lose to layers declared later, regardless of selector specificity within them — giving you a deliberate, name-based way to organize precedence instead of fighting specificity wars.

MDN — Cascade, specificity, and inheritance

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_cascade/Cascade

The complete, official explanation of how the cascade algorithm works.

MDN — @layer

<https://developer.mozilla.org/en-US/docs/Web/CSS/@layer>



- ☐ Keep selectors reasonably simple and shallow.
- ☐ Prefer classes for reusable components over deep descendant chains.
- ☐ Use custom properties for design tokens (colors, spacing, radii).
- ☐ Separate base styles, components, and utilities where useful.
- ☐ Avoid excessive use of !important.
- ☐ Use consistent spacing, typography, and breakpoint scales.
- ☐ Document unusual or non-obvious layout decisions.

BEM-Style Naming

bem-example.css

CSS

```
.card {}  
.card__title {}  
.card__button {}  
.card--featured {}
```

Block, Element, Modifier naming makes relationships explicit purely from the class name: `.card__title` is clearly part of `.card`, and `.card--featured` is clearly a variant of it.



One methodology among several

BEM is one popular naming convention. Utility-first approaches (like Tailwind's) and other component-scoped architectures can work equally well when applied consistently across a whole codebase — the value comes from consistency, not from any one “correct” system.

BEM — Official methodology site

<https://en.bem.info/methodology/>

The original, official documentation for BEM naming.

MDN — Organizing your CSS

https://developer.mozilla.org/en-US/docs/Learn/CSS/Building_blocks/Organizing

MDN's guidance on structuring CSS for larger projects.

Accessibility

- ☐ Maintain a visible keyboard focus indicator at all times.
- ☐ Never rely on color alone to convey meaning or state.
- ☐ Keep text contrast at a readable level against its background.
- ☐ Support browser zoom and responsive, reflowable text.
- ☐ Respect prefers-reduced-motion for all non-essential animation.
- ☐ Use semantic HTML for meaning — CSS cannot fix bad markup.

Performance

- ☐ Remove unused CSS where practical to reduce parse and paint cost.
- ☐ Avoid unnecessarily large, render-blocking stylesheets.
- ☐ Optimize images independently — CSS sizing doesn't reduce file size.
- ☐ Keep layout and animation logic understandable and predictable.
- ☐ Prefer efficient animation properties (transform, opacity).

Security

CSS is not a general security boundary. Real security issues almost always involve HTML injection, JavaScript, server configuration, dependencies, or deployment systems — not the stylesheet itself.



A rare exception worth knowing

Highly specific CSS attack techniques (like CSS-based input-value exfiltration via attribute selectors) exist in security research, but they target unusual or misconfigured setups. For everyday development, treat CSS as presentation-only and focus security effort on your HTML output and server-side handling.

W3C WAI — Web Accessibility Initiative

<https://www.w3.org/WAI/>

Official accessibility guidelines and contrast/motion standards.

web.dev — CSS performance

<https://web.dev/fast/#optimize-your-css>

Google's guidance on writing and shipping efficient CSS.



index.html

HTML

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Study Hub</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="site-header">
    <a class="logo" href="#">Study Hub</a>
    <nav aria-label="Primary">
      <a href="#courses">Courses</a>
      <a href="#about">About</a>
    </nav>
  </header>
  <main>
    <section class="hero">
      <h1>Learn web development.</h1>
      <p>Build practical skills one project at a time.</p>
    </section>
    <section id="courses" class="cards">
      <article class="card"><h2>HTML</h2><p>Structure.</p></article>
      <article class="card"><h2>CSS</h2><p>Presentation.</p></article>
      <article class="card"><h2>JavaScript</h2><p>Behavior.</p></article>
    </section>
  </main>
</body>
</html>
```

style.css

CSS

```
:root {
  --primary: #2563eb;
  --surface: #fff;
  --text: #172033;
  --border: #d9dee7;
}

*, ::before, ::after { box-sizing: border-box; }

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: var(--text);
  background: #f6f8fb;
  line-height: 1.6;
}
```

```

.site-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
  padding: 1rem 5%;
  background: var(--surface);
  border-bottom: 1px solid var(--border);
}

.cards {
  width: min(92%, 1100px);
  margin: 0 auto 4rem;
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}

.card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 16px;
  padding: 1.25rem;
}

```



Notice what each rule is doing

Custom properties define the design tokens once at the top; box-sizing: border-box is reset globally; the header uses Flexbox for its one-dimensional row; and the cards section uses Grid's auto-fit/minmax pattern for a fully responsive layout with zero media queries.



Responsive Profile Card

Profile image, name, role, bio, and buttons; Flexbox alignment, hover/focus states, fluid width.

Product Landing Page

Header, hero, features, pricing, FAQ, and footer; Grid for cards, Flexbox for navigation, mobile-first.

Admin Dashboard

Sidebar, top bar, statistic cards, and a table; a Grid page shell with responsive sidebar behavior.

Login / Register UI

Labels, inputs, and validation states; accessible focus styles; a responsive form layout.



Build with constraints first

Try each project mobile-first and with a self-imposed limit (no fixed pixel widths, no more than 3 custom properties for color) before allowing yourself full freedom. Constraints force you to actually use Flexbox/Grid/clamp() instead of falling back to old habits.



Symptom	Where to look
Horizontal scroll appears	Inspect fixed widths, padding, long unbreakable content, and overflow.
z-index seems ignored	Inspect stacking contexts on ancestor elements.
A flex item won't shrink	Check min-width/min-height and overflow — min-width: 0 often fixes it.
Grid is too wide / overflows	Check minmax() minimums and long unbreakable content inside a track.
Unexpected extra margin	Check margin collapsing and the surrounding formatting context.
A rule isn't applying at all	Check the selector, specificity, cascade layer, and source order.
Mobile layout breaks	Check the viewport meta tag, fixed dimensions, overflow, and media queries.

A Reliable DevTools Workflow

- ☐ Inspect the element in question.
- ☐ Check every matched CSS rule, in cascade order.
- ☐ Look specifically for crossed-out (overridden) declarations.
- ☐ Check the Computed panel for the final resolved value.
- ☐ Toggle properties live to test hypotheses instantly.
- ☐ Use responsive/device viewport testing for layout bugs.
- ☐ Use performance tools when rendering or scrolling feels slow.

Chrome DevTools — CSS features reference

<https://developer.chrome.com/docs/devtools/css>

Official documentation for inspecting and editing CSS live.

Firefox DevTools — CSS Grid Inspector

https://firefox-source-docs.mozilla.org/devtools-user/page_inspector/how_to/examine_grid_layouts/

Official guide to visually debugging Grid layouts.



TEXT

color · font-family · font-size · font-weight · line-height

SPACING

margin · padding · gap

SIZING

width · height · min-* · max-*

BOX

border · border-radius · box-shadow

FLEX

display:flex · justify-content · align-items · flex · gap

GRID

display:grid · grid-template-columns · grid-template-rows · gap

POSITION

position · inset · z-index

RESPONSIVE

@media (min-width / max-width / prefers-color-scheme / prefers-reduced-motion)

STATES

:hover · :focus-visible · :checked · :disabled

GENERATED CONTENT

::before · ::after

MOTION

transition · transform · @keyframes · animation

REUSABLE VALUES & FUNCTIONS

--custom-property · var() · calc() · min() · max() · clamp() · minmax()

Q1 What is CSS and why is it separated from HTML?

A Cascading Style Sheets handle presentation, kept separate from HTML's structure so content and design can change independently.

Q2 What is the difference between class and ID selectors?

A A class can be reused on many elements; an id must be unique per document and carries higher specificity.

Q3 Explain the box model.

A Every box is made of content, then padding, then border, then margin, from the inside out.

Q4 What does box-sizing: border-box do?

A It makes declared width/height include padding and border, instead of adding them on top.

Q5 What is the difference between display:none and visibility:hidden?

A display:none removes the element from layout entirely; visibility:hidden hides it but keeps its layout space.

Q6 Explain relative, absolute, fixed, and sticky.

A relative offsets in place; absolute positions against a containing block, out of flow; fixed anchors to the viewport; sticky switches from relative to fixed at a scroll threshold.

Q7 When should Flexbox be used instead of Grid?

A When you only need to arrange items along one dimension, a single row or column, with flexible sizing.

Q8 What does 1fr mean?

A One fractional unit of the remaining free space in a Grid container, shared proportionally among all fr tracks.

Q9 What is a media query?

A A conditional CSS block that applies rules only when a feature like viewport width or color-scheme preference matches.

Q10 Why is mobile-first useful?

A It forces a simple, functional base layout first, then progressively adds complexity only where screen space allows.

Q11 What is a pseudo-class?

A A selector suffix that matches an element based on state or position, like :hover or :nth-child().

Q12 What is a pseudo-element?

A A selector suffix that targets a generated or sub-part of an element, like ::before or ::first-line.

Q13 How do CSS variables work?

A Custom properties store a reusable value that can be read with var() and can change per element, media query, or at runtime.

Q14 What factors affect the cascade?

A Origin and importance, cascade layers, selector specificity, and finally source order.

Q15 Why should !important be used carefully?

A It overrides normal cascade resolution, making future overrides harder and often masking the real specificity problem.

Q16 How does CSS support accessibility?

A Through visible focus styles, sufficient color contrast, respecting motion preferences, and not hiding meaningful content.

Q17 What does prefers-reduced-motion do?

A It detects a user's OS-level preference for less animation, letting you shorten or disable non-essential motion.



Final Checklist

Confirm mastery before moving on

- ☐ Understand selectors, combinators, and the cascade.
- ☐ Master the box model and box-sizing.
- ☐ Practice typography, spacing, and vertical rhythm.
- ☐ Build layouts confidently with Flexbox.
- ☐ Build two-dimensional layouts confidently with Grid.
- ☐ Write genuinely mobile-first responsive CSS.
- ☐ Use accessible focus and UI states everywhere.
- ☐ Use transitions and animations responsibly, respecting motion preferences.
- ☐ Use custom properties and modern CSS functions.
- ☐ Debug confidently with browser DevTools.
- ☐ Build at least three complete responsive projects.
- ☐ Combine semantic HTML and CSS well before moving deeply into JavaScript frameworks.

Recommended learning order

Syntax → selectors → cascade/specificity → box model → typography → display/positioning → Flexbox → Grid → responsive design → states → animation → variables/functions → architecture → accessibility/performance → projects.



CSS Timeline & History

From one style sheet proposal to dozens of living modules

- 1994–1996**
Håkon Wium Lie proposes Cascading HTML Style Sheets; CSS Level 1 becomes a W3C Recommendation in 1996.
- 1998**
CSS2 is published, adding positioning, z-index, and media types.
- 2000s**
Browser inconsistency stalls a single CSS3 release; the spec is split into independent, separately versioned **modules**.
- 2009–2012**
Flexbox is specified and gradually implemented across browsers, solving decades of vertical-centering pain.
- 2017**
CSS Grid Layout ships in all major browsers, enabling true two-dimensional layout for the first time.
- 2020s**
Container queries, cascade layers (@layer), :has(), and wide-gamut colors (oklch) bring component-level responsiveness and easier architecture.
- Today**
CSS is developed as many independent, continuously evolving modules under the W3C CSS Working Group — there is no single “CSS4.”

W3C CSS Working Group — current work

<https://www.w3.org/Style/CSS/current-work>

The live index of every active CSS specification module.

MDN — CSS: A history

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_history

Mozilla's account of how CSS evolved from CSS1 into today's modular system.



Official Resources & Further Reading

Bookmark these — they are the sources this course draws from

Specifications

W3C CSS Working Group — current work

<https://www.w3.org/Style/CSS/current-work>

The official, authoritative index of every CSS specification module.

CSSWG Editor's Drafts

<https://drafts.csswg.org/>

The bleeding-edge working drafts browsers implement from next.

Learning & Reference

MDN Web Docs — CSS

<https://developer.mozilla.org/en-US/docs/Web/CSS>

The most widely used CSS reference and tutorial hub among professional developers.

web.dev by Google — Learn CSS

<https://web.dev/learn/css>

A structured, modern course covering the full CSS layout and styling model.

CSS-Tricks

<https://css-tricks.com/>

Long-running, widely bookmarked practical guides, especially for Flexbox and Grid.

Layout Deep Dives

Grid by Example

<https://gridbyexample.com/>

Example-driven Grid reference maintained by a CSS Grid spec editor.

Flexbox Froggy

<https://flexboxfroggy.com/>

A free, well-known interactive game for practicing Flexbox properties.

Accessibility & Tools

WebAIM Contrast Checker

<https://webaim.org/resources/contrastchecker/>

Verify text/background color contrast against WCAG thresholds.

Can I Use

<https://caniuse.com/>

Real-time browser support tables for any CSS feature.



A note on authenticity

Every link in this guide points to the primary, official source for that topic — the W3C CSS Working Group itself, the browser vendor's own documentation (MDN, web.dev), or a recognized subject-matter authority (Rachel Andrew for Grid, WebAIM for accessibility). Bookmark this page as your jumping-off point.

You've completed the course.

Structure with HTML. Style with CSS. Bring it to life with JavaScript. Build responsively, accessibly, and maintainably — every single time.

HTML • CSS • JAVASCRIPT